https://github.com/KarimaR23/HeyNeighbor.git

Oct 27th 2026



Oct 29th 2026



Data Dictionary

role

field	description	data type	key	required

role_id	unique identifier for the role	integer	primary key	yes
role_name	role name; unique	string(50)	none (unique)	yes
description	what the role is allowed to do	text	none	no
is_system_role	true for the five built-in roles; they cannot be deleted	boolean	none	yes

permission

field	description	data type	key	required
permission_id	unique identifier for the permission	integer	primary key	yes
permission_code	machine-readable permission code; unique	string(100)	none (unique)	yes
permission_area	group the permission belongs to (listings, moderation, support, administration, ...)	string(40)	none	yes
is_admin_only	true if the permission may be granted only to the administrator role	boolean	none	yes
description	plain-language description of the permission	text	none	no

role_permission

field	description	data type	key	required
role_id	role that is granted the permission	integer	primary key / foreign key → role	yes
permission_id	permission granted to the role	integer	primary key / foreign key → permission	yes

user_account

field	description	data type	key	required

user_id	unique identifier for the account	integer	primary key	yes
role_id	the account's single system role, chosen at sign-up. it stays the same unless an administrator changes it (for example after a role_change support request)	integer	foreign key → role	yes
email	sign-in email address; unique	string(255)	none (unique)	yes
password_hash	hashed password (never plain text). replace with an auth provider uid if an external auth service is used	string(255)	none	yes
account_status	pending, active, deactivated, suspended, or banned	enum(string)	none	yes
email_verified	whether the email address has been verified; default false	boolean	none	yes

display_name	name shown to other users	string(100)	none	yes
phone	contact phone number (private)	string(20)	none	no
avatar_image	profile image uploaded by the user. the uploaded file is kept in file storage and this field holds the reference to it; empty if no image was uploaded	string(500)	none	no
bio	short self-description	text	none	no
city	city of the user's home area	string(100)	none	no
state	state or region	string(50)	none	no
postal_code	zip/postal code, used as the default search center (the application requires it for community users)	string(10)	none	no
latitude	home-area latitude for distance search	decimal(9,6)	none	no
longitude	home-area longitude for	decimal(9,6)	none	no

	distance search			
default_search_radius_miles	preferred maximum search distance	integer	none	no
status_changed_by_user_id	administrator who last changed account_status	integer	foreign key → user_account	no
status_changed_at	when account_status was last changed	datetime	none	no
status_reason	reason given for the last status change (suspension, ban, deactivation)	text	none	no
created_by_user_id	administrator who created the account (staff accounts only); empty for self-registered users	integer	foreign key → user_account	no
created_at	when the account was created	datetime	none	yes
last_login_at	most recent successful sign-in	datetime	none	no
updated_at	when the account row	datetime	none	yes

	was last changed			

organization

field	description	data type	key	required
organization_id	unique identifier for the organization	integer	primary key	yes
owner_user_id	community partner account that owns the organization; unique	integer	foreign key → user_account	yes
org_name	organization name	string(150)	none	yes
org_type	business, nonprofit, church, food_pantry, shelter, community_org, or other	enum(string)	none	yes
description	about the organization	text	none	no
services_offered	summary of services and assistance provided	text	none	no
operating_hours	operating hours as entered by the organization (for example "mon-fri 9am-5pm; sat 10am-2pm")	text	none	no
contact_email	public contact email	string(255)	none	no
contact_phone	public contact phone	string(20)	none	no
website_url	organization website	string(255)	none	no
street_address	street address	string(200)	none	yes
city	city	string(100)	none	yes
state	state or region	string(50)	none	yes
postal_code	zip/postal code	string(10)	none	yes

latitude	location latitude	decimal(9,6)	none	no
longitude	location longitude	decimal(9,6)	none	no
registration_number	optional ein or registration number used for approval review	string(50)	none	no
approval_status	pending, approved, rejected, or deactivated	enum(string)	none	yes
reviewed_by_user_id	administrator who approved or rejected the registration	integer	foreign key → user_account	no
reviewed_at	when the registration was reviewed	datetime	none	no
rejection_reason	reason given if the registration was rejected	text	none	no
created_at	when the organization profile was created	datetime	none	yes
updated_at	when the organization profile was last changed	datetime	none	yes

category

field	description	data type	key	required
category_id	unique identifier for the category	integer	primary key	yes
parent_category_id	parent category, if categories are organized in groups	integer	foreign key → category	no
name	category name; unique	string(80)	none (unique)	yes
description	what belongs in the category	text	none	no
display_order	sort order in menus and filters	integer	none	yes
is_active	inactive categories cannot be chosen for new listings	boolean	none	yes

fulfillment_method

field	description	data type	key	required
method_id	unique identifier for the method	integer	primary key	yes
method_name	method name; unique	string(40)	none (unique)	yes
description	meaning of the method	text	none	no
is_active	inactive methods cannot be chosen for new listings	boolean	none	yes

listing

field	description	data type	key	required
listing_id	unique identifier for the listing	integer	primary key	yes
owner_user_id	account that created and owns the listing	integer	foreign key → user_account	yes
organization_id	set when a community partner posts on behalf of its organization	integer	foreign key → organization	no
category_id	the listing's category (food, shelter, utilities, services, ...)	integer	foreign key → category	yes
listing_status	active, reserved, completed, inactive, hidden,	enum(string)	none	yes

	removed, or expired. only active listings appear in the public feed and search			
fulfillment_method_id	pickup, drop-off, or both	integer	foreign key → fulfillment_method	yes
title	listing title	string(150)	none	yes
description	full description	text	none	yes
listing_type	gift or trade	enum(string)	none	yes
offering_kind	good or service	enum(string)	none	yes
trade_wanted	what the owner would like in return; required when listing_type is trade	text	none	no
quantity_available	quantity remaining (mainly for community partner resources)	integer	none	no
quantity_unit	unit for the quantity (boxes, meals, hours, ...)	string(30)	none	no
eligibility_info	eligibility requirements (community partner listings)	text	none	no
availability_notes	schedule or availability details	text	none	no

pickup_dropoff_notes	pickup or drop-off instructions	text	none	no
listing_images	images uploaded by the user for the listing. each uploaded file is kept in file storage and this field holds the list of references to them, in display order (for example a json array); empty when no images were uploaded	text	none	no
city	city where the item or service is available	string(100)	none	no
postal_code	zip/postal code	string(10)	none	yes
latitude	latitude used for distance search	decimal(9,6)	none	yes
longitude	longitude used for distance search	decimal(9,6)	none	yes
available_from	when the listing becomes available	datetime	none	no
expires_at	when the listing	datetime	none	no

	automatically expires			
created_at	when the listing was created	datetime	none	yes
updated_at	when the listing was last changed	datetime	none	yes
published_at	when the listing was published to the feed	datetime	none	no
completed_at	when the listing's exchange was completed	datetime	none	no

request

field	description	data type	key	required
request_id	unique identifier for the request	integer	primary key	yes
listing_id	listing being requested (or the trade listing a trade is proposed on)	integer	foreign key → listing	yes
requester_user_id	user making the request	integer	foreign key → user_account	yes
request_status	pending, approved (accepted, for trades), declined (rejected, for trades), withdrawn, fulfilled, or expired	enum(string)	none	yes
message	introductory note from the requester	text	none	no

quantity_requested	amount requested, where the listing has a quantity	integer	none	no
offered_listing_id	trade proposals only: the requester's own listing offered in exchange	integer	foreign key → listing	no
offered_description	trade proposals only: written description of the offer when it is not an existing listing	text	none	no
requested_at	when the request was made	datetime	none	yes
responded_at	when the owner approved or declined	datetime	none	no
responded_by_user_id	account that approved or declined (owner or partner staff)	integer	foreign key → user_account	no
decline_reason	reason given if declined	text	none	no

exchange

field	description	data type	key	required
exchange_id	unique identifier for the exchange	integer	primary key	yes
listing_id	listing being exchanged	integer	foreign key → listing	yes
provider_user_id	user giving the item or service (the listing owner)	integer	foreign key → user_account	yes

receiver_user_id	user receiving it (the requester or trade proposer)	integer	foreign key → user_account	yes
request_id	the approved request or trade proposal this exchange comes from; unique	integer	foreign key → request	yes
fulfillment_method_id	agreed method (pickup or drop-off)	integer	foreign key → fulfillment_method	yes
quantity	quantity covered by this exchange	integer	none	no
scheduled_at	agreed date and time of the handoff	datetime	none	no
meeting_notes	agreed location or arrangement notes	text	none	no
exchange_status	reserved, completed, cancelled, or disputed	enum(string)	none	yes
created_at	when the exchange (reservation) was created	datetime	none	yes
completed_at	when the exchange was marked completed	datetime	none	no
completed_by_user_id	participant who marked it completed	integer	foreign key → user_account	no

cancelled_at	when the exchange was cancelled	datetime	none	no
cancel_reason	reason for cancellation	text	none	no

review

field	description	data type	key	required
review_id	unique identifier for the review	integer	primary key	yes
exchange_id	completed exchange being reviewed	integer	foreign key → exchange	yes
reviewer_user_id	user writing the review	integer	foreign key → user_account	yes
reviewee_user_id	user being reviewed	integer	foreign key → user_account	yes
overall_rating	overall rating, 1 to 5	integer	none	yes
reliability_rating	reliability rating, 1 to 5	integer	none	no
friendliness_rating	friendliness rating, 1 to 5	integer	none	no
comment	written feedback	text	none	no
review_status	visible, hidden, or removed (set by moderation)	enum(string)	none	yes
created_at	when the review was submitted	datetime	none	yes
updated_at	when the review was last changed	datetime	none	yes

conversation

field	description	data type	key	required
conversation_id	unique identifier for the conversation	integer	primary key	yes

conversation_type	inquiry, request, exchange, or support	enum(string)	none	yes
listing_id	listing the conversation is about	integer	foreign key → listing	no
request_id	request or trade proposal the conversation is about	integer	foreign key → request	no
exchange_id	exchange the conversation is about	integer	foreign key → exchange	no
support_request_id	support request the conversation is about	integer	foreign key → support_request	no
is_locked	true if no new messages may be sent (e.g. after moderation)	boolean	none	yes
created_at	when the conversation started	datetime	none	yes
last_message_at	time of the newest message; used to sort the inbox	datetime	none	no
participant_user_ids	list of the user ids taking part in the conversation (two or more); a user appears only once	text	none	yes
participant_status	per-participant list entry: user id, when they	text	none	yes

	joined, when they last read the conversation (unread counts), and whether they archived it			
messages	ordered list of the messages in the conversation. each entry holds: message number (1, 2, 3…), sender user id, message text, time sent, and status (visible or removed by moderation)	text	none	no

notification

field	description	data type	key	required
notification_id	unique identifier for the notification	integer	primary key	yes
user_id	user the notification is for	integer	foreign key → user_account	yes
notification_type	type such as new_message, request_received, trade_proposed, exchange_completed, report_update	enum(string)	none	yes
title	short headline	string(150)	none	yes
body	notification text	text	none	no
related_entity_type	kind of record the notification points to (listing, exchange, …)	string(40)	none	no

related_entity_id	identifier of the record the notification points to (not a database fk)	integer	none	no
is_read	whether the user has read it	boolean	none	yes
created_at	when the notification was created	datetime	none	yes
read_at	when the user read it	datetime	none	no

favorite

field	description	data type	key	required
user_id	user who saved the listing	integer	primary key / foreign key → user_account	yes
listing_id	the saved listing	integer	primary key / foreign key → listing	yes
created_at	when the listing was saved	datetime	none	yes

recent_view

field	description	data type	key	required
user_id	user who viewed the listing	integer	primary key / foreign key → user_account	yes
listing_id	the viewed listing	integer	primary key / foreign key → listing	yes
last_viewed_at	most recent time the user viewed it (updated on repeat views)	datetime	none	yes

report

field	description	data type	key	required
report_id	unique identifier for the report	integer	primary key	yes
reporter_user_id	user who filed the report	integer	foreign key → user_account	yes
report_type	listing, user, message, review, or exchange	enum(string)	none	yes
reported_listing_id	reported listing (when report_type = listing)	integer	foreign key → listing	no
reported_user_id	reported user (when report_type = user)	integer	foreign key → user_account	no
reported_conversation_id	conversation holding the reported message (when report_type = message)	integer	foreign key → conversation	no
reported_message_number	number of the reported message inside that conversation (when report_type = message)	integer	none	no
reported_review_id	reported review (when	integer	foreign key → review	no

	report_type = review)			
reported_exchange_id	reported exchange (when report_type = exchange)	integer	foreign key → exchange	no
routed_to_role_id	role that should review the report (moderator, support coordinator, administrator)	integer	foreign key → role	no
case_id	moderation case the report was attached to	integer	foreign key → moderation_case	no
reason	harassment, fraud, unsafe, inappropriate, spam, no_show, other	enum(string)	none	yes
description	details written by the reporter	text	none	no
report_status	open, in_review, resolved, or dismissed	enum(string)	none	yes
created_at	when the report was filed	datetime	none	yes
resolved_at	when the report was	datetime	none	no

	resolved or dismissed			

moderation_case

field	description	data type	key	required
case_id	unique identifier for the case	integer	primary key	yes
assigned_moderator_id	community moderator handling the case	integer	foreign key → user_account	yes
subject_user_id	user the case is mainly about	integer	foreign key → user_account	no
title	short case title	string(150)	none	yes
case_status	open, under_review, escalated, or closed	enum(string)	none	yes
findings	moderator's recorded findings	text	none	no
resolution	final outcome and explanation of the action taken	text	none	no
action_type	action the moderator took: warning, listing_hidden, listing_removed, review_removed, message_removed, restriction_applied, or no_action. a user warning or	enum(string)	none	no

	restriction applies to subject_user_id			
target_listing_id	listing the action applies to	integer	foreign key → listing	no
target_review_id	review the action applies to	integer	foreign key → review	no
target_conversation_id	conversation holding the message the action applies to	integer	foreign key → conversation	no
target_message_number	number of the message inside that conversation the action applies to	integer	none	no
action_taken_at	when the action was taken	datetime	none	no
escalated_to_role_id	role the case was escalated to (support coordinator or administrator)	integer	foreign key → role	no
escalation_reason	why the case was escalated	text	none	no
escalated_at	when the case was escalated	datetime	none	no
opened_at	when the case was opened	datetime	none	yes
updated_at	when the case was last changed	datetime	none	yes
closed_at	when the case was closed	datetime	none	no

user_restriction

field	description	data type	key	required

restriction_id	unique identifier for the restriction	integer	primary key	yes
case_id	case the restriction came from	integer	foreign key → moderation_case	yes
user_id	user who is restricted	integer	foreign key → user_account	yes
imposed_by_user_id	moderator who imposed the restriction	integer	foreign key → user_account	yes
restriction_type	no_listings, no_messages, or no_exchanges	enum(string)	none	yes
reason	why the restriction was imposed	text	none	yes
starts_at	when the restriction begins	datetime	none	yes
ends_at	when the restriction ends; always set, because restrictions are temporary	datetime	none	yes
lifted_at	set if the restriction was lifted early	datetime	none	no
created_at	when the restriction was recorded	datetime	none	yes

support_request

field	description	data type	key	required
support_request_id	unique identifier for the support request	integer	primary key	yes

requester_user_id	user or partner asking for help	integer	foreign key → user_account	yes
assigned_coordinator_id	support coordinator handling the request	integer	foreign key → user_account	no
related_listing_id	listing the problem concerns	integer	foreign key → listing	no
related_exchange_id	exchange the problem concerns (required when request_type is exchange_problem or dispute)	integer	foreign key → exchange	no
request_type	exchange_problem, account_problem, partner_assistance, dispute, role_change, other	enum(string)	none	yes
requested_role_id	only for role_change: the role the user is asking to be changed to (required for that type). only an administrator can approve it	integer	foreign key → role	no
subject	short summary	string(150)	none	yes
description	full description of the problem	text	none	yes
status	open, in_progress, waiting_on_user, escalated, resolved, or closed	enum(string)	none	yes

priority	low, normal, high, or urgent	enum(string)	none	yes
dispute_type	only for disputes: no_show, not_as_described, unavailable, communication, or other	enum(string)	none	no
recommended_resolution	support coordinator's recommended resolution (disputes)	text	none	no
resolution_summary	how the issue was resolved (the final outcome for disputes)	text	none	no
escalated_to_role_id	role the request was escalated to (moderator or administrator)	integer	foreign key → role	no
escalation_reason	why the request was escalated	text	none	no
escalated_at	when the request was escalated	datetime	none	no
notes	ordered list of the responses and documentation entries on the request. each entry holds: author user id, note text, whether it is internal (only staff may see it), and time written	text	none	no

created_at	when the request was created	datetime	none	yes
updated_at	when the request was last changed	datetime	none	yes
resolved_at	when the issue was resolved	datetime	none	no
closed_at	when the request was closed	datetime	none	no

system_setting

field	description	data type	key	required
setting_id	unique identifier for the setting	integer	primary key	yes
setting_key	setting name, e.g. max_search_radius_miles; unique	string(100)	none (unique)	yes
setting_value	current value	string(500)	none	yes
value_type	string, integer, boolean, or decimal	enum(string)	none	yes
description	what the setting controls	text	none	no
updated_by_user_id	administrator who last changed it	integer	foreign key → user_account	no
updated_at	when it was last changed	datetime	none	yes

Database Business Rules

A. Accounts, roles and permissions

Id	Rule	Entities	Enforced

Br-01	A user email must be unique (compare case-insensitively).	User_account	Db
Br-02	Every account has exactly one role, chosen from the five system roles. The role chosen at sign-up stays the same; only an administrator can change it (br-77).	User_account, role	Db (fk, not null)
Br-03	Only an administrator can create community moderator, support coordinator, or administrator accounts; self-registration is limited to community user and community partner.	User_account	App
Br-04	Passwords are stored only as hashes; plain-text passwords are never stored.	User_account	App
Br-05	Only active accounts can sign in. Pending, deactivated, suspended, and banned accounts are refused.	User_account	App
Br-06	Profile information (display name, avatar, bio, city, location, search radius) is stored on the account. Public profile screens return only display name, avatar, bio, city, and reputation; email, phone, password hash, and status are never returned to other users.	User_account	App
Br-07	Permissions are granted to roles, not to individual users. A role cannot be granted the same permission twice.	Role_permission	Db (composite pk)
Br-08	Every change to account_status records who made it, when, and why (status_changed_by_user_id, status_changed_at, status_reason).	User_account	App
Br-09	Ratings, reviews, and listings of a banned or deactivated user stay in the database for history; they are hidden, not deleted.	User_account, listing, review	App

B. Community Partners

Id	Rule	Entities	Enforced

Br-10	A community partner account owns exactly one organization, and an organization has exactly one owner account.	Organization	Db (unique owner_user_id)
Br-11	A partner organization must be approved by an administrator before its listings become publicly visible.	Organization, listing	App
Br-12	Rejecting an organization requires a rejection reason; approval/rejection records who decided and when.	Organization	App
Br-13	A listing may reference an organization only if the listing owner is that organization's owner account.	Listing	App
Br-14	A partner can view and manage only its own organization's listings, requests, history, and feedback.	Listing, request, review	App
Br-15	Operating hours are kept as text on the organization profile (operating_hours) and shown to users as entered.	Organization	App

C. Listings, categories and statuses

Id	Rule	Entities	Enforced
Br-16	A listing belongs to exactly one owner.	Listing	Db (fk, not null)
Br-17	A listing must have a title, description, type (gift or trade), kind (good or service), fulfillment method, status, and a location (postal code, latitude, longitude).	Listing	Db (not null)
Br-18	Every listing belongs to exactly one category (category_id), chosen from the active categories.	Listing, category	Db (fk, not null)
Br-19	Only active categories can be chosen for new listings. Deactivating a category does not change existing listings.	Category	App

Br-20	A listing's status must be one of active, reserved, completed, inactive, hidden, removed, or expired.	Listing	Db (check)
Br-21	Only listings with status active appear in the community feed and search.	Listing	App
Br-22	A trade listing must describe what the owner wants in return (trade_wanted).	Listing	App
Br-23	Quantity_available cannot be negative. When it reaches 0, the listing is no longer requestable.	Listing	Db (check) + app
Br-24	A listing whose expires_at has passed is set to expired and removed from the active feed.	Listing	App (scheduled job)
Br-25	Only the owner (or an administrator/moderator acting in their role) can edit, deactivate, or remove a listing.	Listing	App
Br-26	Latitude must be between -90 and 90, longitude between -180 and 180.	Listing, user_account, organization	Db (check)
Br-27	Search distance cannot exceed the administrator-configured maximum (max_search_radius_miles in system_setting).	User_account, system_setting	App
Br-28	A listing can have a limited number of photos (configurable, default 8). Images uploaded for the listing are saved in file storage, and their links are recorded in the listing's listing_images field.	Listing	App

D. Requests, trades, exchanges (reservations)

Id	Rule	Entities	Enforced
Br-29	A user cannot request or propose a trade on their own listing.	Request	App
Br-30	A user may have only one open (pending) request per listing.	Request	Db (partial unique) or app

Br-31	A user can submit many requests and trade proposals across different listings.	Request	— (1:m by design)
Br-32	Only active listings can receive new requests or trade proposals.	Request	App
Br-33	A request can include a trade offer only when the requested listing is a trade listing. An offered listing must belong to the requester and cannot be the same as the requested listing.	Request	App + db (check requested ≠ offered)
Br-34	A request on a trade listing must contain either an offered listing (offered_listing_id) or a written offer (offered_description).	Request	App
Br-35	A listing can have only one active (reserved) exchange at a time, unless it is a quantity-based partner listing, where the total reserved quantity cannot exceed quantity_available.	Exchange	App (db partial unique for single-item listings)
Br-36	Every exchange comes from exactly one approved request (or trade proposal).	Exchange	Db (fk, not null)
Br-37	A request can become at most one exchange.	Exchange	Db (unique request_id)
Br-38	Creating a reserved exchange sets the listing status to reserved. For an accepted trade, both listings are set to reserved.	Exchange, listing	App
Br-39	Provider and receiver of an exchange must be different users.	Exchange	Db (check)
Br-40	Only the provider or receiver of an exchange can mark it completed. Completion sets completed_at, sets the listing to completed, and removes it from the active feed.	Exchange, listing	App
Br-41	If an exchange is cancelled, its listing returns to active (unless it expired or was removed), and a reason is recorded.	Exchange, listing	App
Br-42	A completed exchange cannot be changed back to reserved.	Exchange	App

E. Ratings and reviews

Id	Rule	Entities	Enforced
Br-43	A review can be written only for an exchange with status completed.	Review	App
Br-44	Only the two participants of the exchange can review, and only each other (reviewer ≠ reviewee).	Review	App + db (check)
Br-45	Each participant can leave at most one review per exchange.	Review	Db (unique exchange_id + reviewer_user_id)
Br-46	Ratings are whole numbers from 1 to 5.	Review	Db (check)
Br-47	A user's reputation score is calculated from visible reviews and is not stored as a separate field.	Review	App (query)
Br-48	Reviews removed by a moderator are kept with status removed and excluded from reputation.	Review	App

F. Messaging, notifications, favorites

Id	Rule	Entities	Enforced
Br-49	A message is stored inside exactly one conversation and has exactly one sender, who must be one of its participants.	Conversation	App
Br-50	Only participants of a conversation can read or send messages in it.	Conversation	App
Br-51	A user appears at most once in a conversation's participant list.	Conversation	App
Br-52	Users with an active no_messages restriction cannot send messages. Locked conversations accept no new messages.	User_restriction, conversation	App
Br-53	Moderator-removed messages are kept in the conversation with status removed and shown as removed.	Conversation	App

Br-54	A user can favorite a listing only once.	Favorite	Db (composite pk)
Br-55	Recently viewed keeps one row per user and listing and updates its time on repeat views. Only the most recent n (default 50) are kept per user.	Recent_view	Db (composite pk) + app
Br-56	A notification belongs to one user and is visible only to that user.	Notification	App

G. Reports, moderation, support, administration

Id	Rule	Entities	Enforced
Br-57	A report has exactly one reported target, and the filled-in reported_* column must match report_type.	Report	Db (check)
Br-58	A user cannot report themselves.	Report	App
Br-59	A moderation case may group many reports; each report belongs to at most one case.	Report, moderation_case	Db (fk)
Br-60	Closing a moderation case requires recorded findings and a resolution.	Moderation_case	App
Br-61	Moderators can hide or remove content and issue temporary restrictions only. Banning or permanently disabling an account requires an administrator.	Moderation_case, user_restriction, user_account	App
Br-62	A restriction must have a restriction_type and is always temporary: ends_at is required and later than starts_at.	User_restriction	Db (check) + app
Br-63	Every restriction is linked to a case and records the user restricted and the moderator who imposed it. A case records at most one action taken.	User_restriction, moderation_case	Db (fk, not null)
Br-64	A support request has one requester and at most one assigned support coordinator at a time.	Support_request	Db (fk)

Br-65	A support request can only be closed after it has been resolved and a resolution summary exists.	Support_request	App
Br-66	A dispute is a support request with request_type = dispute and must refer to an exchange (related_exchange_id is required).	Support_request	App
Br-67	Only the two participants of the exchange can open a dispute for it (the requester must be the provider or receiver).	Support_request, exchange	App
Br-68	Escalating a moderation case or support request requires the receiving role and a reason (escalated_to_role_id, escalation_reason), records the time, and sets the status to escalated.	Moderation_case, support_request	App
Br-69	Internal support notes (is_internal) are never visible to the requesting user.	Support_request	App
Br-70	System settings keys are unique and each change records the administrator and time.	System_setting	Db (unique) + app
Br-71	Role-based access: moderators see only reported content and related case data; support coordinators see only support-related data; administrators manage the platform through the admin view. What each role may do is defined in 07_role_permission_matrix.Md.	All	App
Br-72	A user's permissions come only from their single role (user_account.Role_id → role_permission → permission). Every protected action checks that permission first.	User_account, role_permission, permission	App
Br-73	A permission with is_admin_only = true can be linked only to the administrator role. Any attempt to grant it to another role is rejected.	Role_permission, permission	App (+ db trigger or check if supported)
Br-74	Administrator-only data (roles, permissions, system settings, category management, account status changes) can be viewed or	Role, permission, role_permission,	App

	changed only by an administrator. Other roles never see the admin view.	system_setting, category	
Br-75	The administrator role does not receive community-participation permissions (create listing, request, trade, review). Administrators act on content through administrative permissions only.	Role_permission	App
Br-76	The five built-in roles cannot be deleted (is_system_role), and the system must always keep at least one active administrator account.	Role, user_account	App
Br-77	Only an administrator can change a user's role (change_user_role, admin-only). Users, moderators and support coordinators cannot change their own or anyone else's role.	User_account, role	App
Br-78	A user who wants a different role submits a support request with request_type = role_change and the wanted role in requested_role_id. It is routed to the administrator (escalated_to_role_id); the decision is saved in resolution_summary, and only if approved does the administrator update user_account.Role_id.	Support_request, user_account, role	App

Modify:

Removed

Audit_log: no requirement needs it. Account status changes are kept on user_account.

Organization_hours: replaced by one operating_hours text field on organization.

Listing_status: replaced by a listing_status list on listing.

Listing_category: replaced by a single category_id on listing.

Escalation: replaced by escalation fields on moderation_case and support_request.

Merged

User_profile → user_account.

Dispute → support_request (request_type = dispute).

Support_note → support_request (notes list).

Trade_proposal → request (offered_listing_id, offered_description).

Listing_photo → listing (listing_images, the uploaded images).

Message and conversation_participant → conversation (messages, participant_user_ids, participant_status).

Moderation_action → moderation_case (action taken is on the case).

Kept as its own table

User_restriction: temporary restrictions on a user (moderation).

Role changes (no new table)

A role is fixed at sign-up. A user asks for a change with a support_request of type role_change (requested_role_id); only an administrator approves it and updates user_account.Role_id.

Trade-offs to know

Messages, participants and notes are stored as lists, so the application (not the database) checks senders, authors and one-user-per-conversation.

Total 20 tables